# CRE

## Context Retrieval Engine

*Product Requirements Document · v0.1 Draft · April 2026*

Built by Porjanya Bordoloi (The Flame)
RGU · AI & Data Science · Guwahati

# 0. The Problem Manifesto

> *"Your prompts explode in size when you include research and notes. No system exists. It's chaos. Every session you either re-explain everything from scratch, or you paste walls of raw context and watch your token budget evaporate."*

This is the #1 unsolved UX problem in production LLM usage. The Reddit thread that inspired CRE surfaced five distinct failure modes, all stemming from the same root: there is no intelligent layer between your knowledge and your token window.

| Failure Mode | What It Actually Costs You |
|---|---|
| Context Rot | Claude understood your architecture at turn 1. By turn 60 it's suggesting approaches you already rejected. |
| Prompt Explosion | Every research note, every decision log, every code snippet goes in raw. Context window fills. Latency spikes. Cost doubles. |
| Signal-to-Noise Collapse | As memory grows, vector retrieval pulls chunks that are semantically similar but structurally irrelevant. Garbage in, garbage out. |
| Manual Overhead | Power users build entire workflows around managing Claude's context. That's cognitive overhead that should go toward actual work. |
| Session Amnesia | No memory between chats. Every new session starts from zero. You re-explain your architecture, your constraints, your decisions — every single time. |

**CRE is the missing pipe between your compiled knowledge and your token window.**

# 1. Product Overview

| | |
|---|---|
| **Project Name** | CRE — Context Retrieval Engine |
| **Version** | v0.1 (Bihu Sprint, April 2026) |
| **Type** | Open-source Python CLI tool |
| **Primary User** | LLM power users, developers, researchers |
| **License** | MIT (open source from day 1) |
| **Builder** | Porjanya Bordoloi (The Flame) |
| **Target Timeline** | 1–2 weeks (Bihu sprint) |
| **Language** | Python 3.10+ |
| **Backend** | Flexible — Anthropic, OpenAI, Ollama (pluggable) |

## 1.1 The One-Line Pitch

*CRE is a composable, model-agnostic CLI tool that sits between your knowledge base and your LLM — compressing, tiering, and injecting only the context that actually matters, so you never waste another token.*

## 1.2 Inspiration Stack

- TunnelVision (Coneja-Chibi) — AI-driven lorebook retrieval; the insight that active retrieval beats passive injection
- Andrej Karpathy's LLM Wiki — knowledge compilation over retrieval; answers compound into the knowledge base
- Reddit r/LocalLLaMA / r/ClaudeAI — five real failure modes documented by power users
- ARIA OS (The Flame's personal intelligence layer) — the real workflow this tool is built to serve

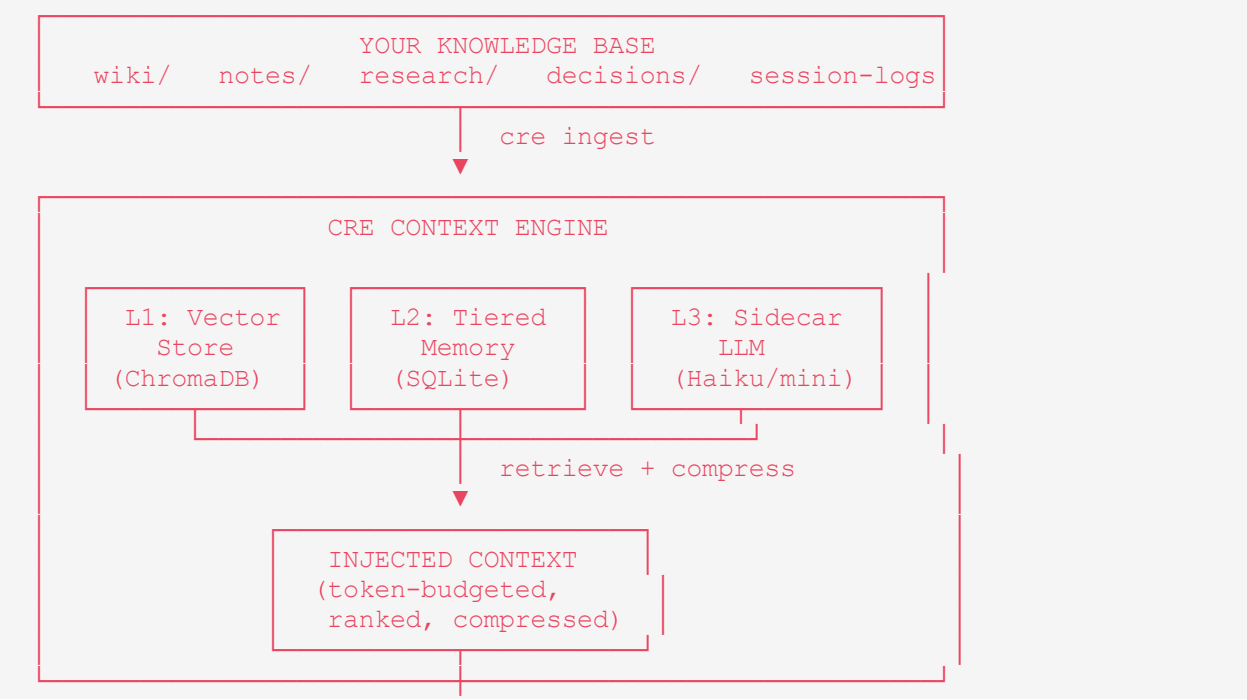
# 2. System Architecture

## 2.1 The Three-Layer Stack

CRE is a full-stack composable context engine. Three layers, each independently useful, all working together:

| Layer | Name | What It Does | Tech |
|---|---|---|---|
| L1 | Vector Store | Embed your wiki/notes. Retrieve only relevant chunks per query. Kill signal-to-noise collapse. | ChromaDB + sentence-transformers |
| L2 | Tiered Memory | Raw facts → summaries → themes. Store at the right tier. Inject at the right level of abstraction. | SQLite + custom tier logic |
| L3 | Sidecar LLM | Small/cheap model compresses and curates context before the main model sees it. Every retrieval costs sidecar tokens, not main-model tokens. | Pluggable: Haiku / GPT-4o-mini / Ollama |

## 2.2 Architecture Diagram (Text)



```
                          │  cre inject
                          ▼

┌──────────────────────────────────────────────────────────┐
│              YOUR MAIN LLM  (Claude / GPT / etc.)        │
│         Receives only relevant, compressed context       │
└──────────────────────────────────────────────────────────┘
```

## 2.3 Data Flow

1. User runs cre ingest <path> — CRE reads markdown files, chunks them, embeds into ChromaDB (L1), and classifies facts into memory tiers (L2).
2. User runs cre retrieve "<query>" — L1 vector search finds relevant chunks. L2 pulls tiered summaries. L3 sidecar LLM compresses and ranks the combined context.
3. User runs cre inject — CRE outputs a token-budgeted, formatted context block ready to paste or pipe into any LLM prompt.
4. User runs cre compress <session_log> — sidecar LLM reads a raw session log and distills it into facts/summaries, filing back into L2 tiers.

## 2.4 Why This Architecture Beats RAG Alone

| | |
|---|---|
| **Pure RAG** | Retrieves semantically similar chunks. Has no concept of why information matters. Read-only — nothing flows back into the store. |
| **CRE L1 + L2 + L3** | L1 retrieves. L2 provides structured abstraction tiers. L3 reasons about what's actually relevant to the current query. Bidirectional — compression results flow back into memory. |
| **TunnelVision** | Brilliant for lorebook/roleplay, tied to SillyTavern, JS-only, no CLI, no general dev workflow support. |
| **CRE** | Model-agnostic, Python, CLI-first, composable layers, built for dev + research + personal knowledge workflows. |

# 3. Feature Specification

## 3.1 MVP Scope (Bihu Sprint — v0.1)

*Rule: If it doesn't solve the 'prompt explosion' problem directly, it does not go into v0.1.*

### Core CLI Commands

| Command | What It Does | MVP Priority |
|---|---|---|
| cre init | Initialise a CRE project in current dir. Creates .cre/ config + stores. | ✅ v0.1 |
| cre ingest <path> | Ingest markdown files/dirs into L1 vector store + L2 memory tiers. | ✅ v0.1 |
| cre retrieve <query> | Retrieve relevant context for a query. Outputs ranked chunks. | ✅ v0.1 |
| cre inject <query> | Retrieve + format a ready-to-paste context block with token budget. | ✅ v0.1 |
| cre compress <file> | Run sidecar LLM on a session log / raw notes. Extract facts → tiers. | ✅ v0.1 |
| cre status | Show store stats: chunks, tiers, token estimates, sidecar config. | ✅ v0.1 |
| cre tui | Launch optional TUI viewer for browsing context live. | 🟡 v0.2 |
| cre lint | Health check: orphan chunks, stale tiers, contradictions. | 🟡 v0.2 |
| cre sync | Auto-ingest from a watched directory (daemon mode). | 🔵 v0.3 |

### 3.2 L1 — Vector Store (ChromaDB)

- Embed markdown files using sentence-transformers (all-MiniLM-L6-v2 default)
- Chunking: sliding window, 512 tokens, 64-token overlap
- Metadata tags on every chunk: source file, domain, created_at, tier_hint
- Query returns top-K chunks with relevance scores
- Local persistence — .cre/vector_store/

### 3.3 L2 — Tiered Memory (SQLite)

- Three tiers stored in .cre/memory.db:
  - Tier 1 — Raw Facts: verbatim extracted facts, decisions, code snippets
  - Tier 2 — Summaries: LLM-compressed summaries of related facts (paragraph-level)
  - Tier 3 — Themes: high-level themes extracted across summaries (bullet-level)

- Inject logic: budget-aware tier selection — themes first (cheapest), then summaries, then raw facts if budget allows
- Manual tier override: cre ingest --tier 2 <file>

### 3.4 L3 — Sidecar LLM

- Pluggable backend via .cre/config.yaml:
  - anthropic: claude-haiku-4-5 (cheapest Anthropic option)
  - openai: gpt-4o-mini
  - ollama: any local model (phi3, mistral, etc.)
- Sidecar tasks: compress session logs, rank retrieved chunks, resolve contradictions between tiers
- Token tracking: every sidecar call logs input/output tokens to .cre/token_log.jsonl
- Fallback: if no sidecar configured, CRE operates in L1+L2 only mode (no compression, still useful)

### 3.5 Token Budget System

- User sets token budget per inject call (default: 2000 tokens)
- CRE fills budget greedily: Tier 3 themes → Tier 2 summaries → Tier 1 facts → raw chunks
- Output includes a token usage summary: 'Injected 1,847 / 2,000 tokens (3 themes, 5 summaries, 2 raw facts)'

# 4. Claude Code Build Guide — CLI Skeleton

Hand this section directly to Claude Code. It is the complete build specification for the v0.1 skeleton.

## 4.1 Project Structure

```
cre/
├── cre/                        # Main package
│   ├── __init__.py
│   ├── cli.py                  # Typer CLI entrypoint — all commands
│   ├── config.py               # Load/save .cre/config.yaml
│   ├── ingestor.py             # File reading, chunking, embedding
│   ├── vector_store.py         # ChromaDB wrapper (L1)
│   ├── memory.py               # SQLite tiered memory (L2)
│   ├── sidecar.py              # Pluggable LLM backend (L3)
│   ├── retriever.py            # Orchestrate L1+L2+L3 retrieval
│   ├── injector.py             # Token-budget context formatting
│   └── tui.py                  # Textual TUI (v0.2, stub for now)
├── tests/
│   ├── test_ingestor.py
│   ├── test_memory.py
│   └── test_retriever.py
├── .cre/                       # Per-project state (gitignored)
│   ├── config.yaml
│   ├── memory.db
│   ├── vector_store/
│   └── token_log.jsonl
├── pyproject.toml
├── README.md
└── CONTRIBUTING.md
```

## 4.2 Dependencies (pyproject.toml)

```
typer[all]>=0.12           # CLI framework
chromadb>=0.5              # L1 vector store
sentence-transformers>=3   # Embeddings
anthropic>=0.25            # Sidecar: Anthropic
openai>=1.0                # Sidecar: OpenAI
ollama>=0.2                # Sidecar: Ollama
textual>=0.60              # TUI (v0.2 stub)
rich>=13                   # Terminal output
pyyaml>=6                  # Config
tiktoken>=0.7              # Token counting
```

## 4.3 Core Module Specs

### cli.py — Command Definitions

Build with Typer. Every command must have a --help string. Commands to implement in v0.1:

- app = typer.Typer(name='cre', help='Context Retrieval Engine')
- @app.command() def init(path: str = '.')
- @app.command() def ingest(path: str, tier: int = 0, domain: str = '')
- @app.command() def retrieve(query: str, top_k: int = 5, budget: int = 2000)
- @app.command() def inject(query: str, budget: int = 2000, format: str = 'markdown')
- @app.command() def compress(file: str, tier: int = 2)
- @app.command() def status()

### config.py — .cre/config.yaml Schema

```
version: '0.1'
sidecar:
  backend: anthropic          # anthropic | openai | ollama | none
  model: claude-haiku-4-5
  api_key_env: ANTHROPIC_API_KEY
embedding:
  model: all-MiniLM-L6-v2
  chunk_size: 512
  chunk_overlap: 64
memory:
  default_inject_budget: 2000  # tokens
  tier_weights: [0.5, 0.35, 0.15]  # themes, summaries, facts
```

### memory.py — SQLite Schema

```
CREATE TABLE memory (
  id          TEXT PRIMARY KEY,
  content     TEXT NOT NULL,
  tier        INTEGER NOT NULL,   -- 1=raw, 2=summary, 3=theme
  domain      TEXT,
  source_file TEXT,
  token_count INTEGER,
  created_at  TEXT,
  updated_at  TEXT,
  tags        TEXT                -- JSON array
);
```

### sidecar.py — Backend Interface

Implement as an abstract base class with three concrete backends:

- class SidecarBackend(ABC): with compress(text: str) -> str and rank(query: str, chunks: list) -> list
- class AnthropicSidecar(SidecarBackend)
- class OpenAISidecar(SidecarBackend)
- class OllamaSidecar(SidecarBackend)
- class NoOpSidecar(SidecarBackend) — passthrough, no LLM calls (for L1+L2 only mode)
- Factory: get_sidecar(config) -> SidecarBackend

### retriever.py — Orchestration Logic

- retrieve(query, budget, config) -> ContextBundle

- Step 1: L1 vector search → top-K chunks with scores
- Step 2: L2 tier fetch → themes + summaries for matching domains
- Step 3: L3 sidecar rank + compress the combined result
- Step 4: Budget-fill: pack ContextBundle greedily by tier priority
- Returns ContextBundle(themes, summaries, facts, raw_chunks, token_count, metadata)

## 4.4 Claude Code Instructions

*Copy this section as your Claude Code session prompt:*

'Build the CRE (Context Retrieval Engine) Python CLI skeleton. Full project structure as specified. Implement all v0.1 CLI commands with Typer. L1: ChromaDB wrapper with sentence-transformers embedding and sliding window chunking. L2: SQLite tiered memory with schema as specified. L3: Pluggable sidecar with Anthropic/OpenAI/Ollama backends using abstract base class pattern. Retriever orchestrates all three layers with token-budget-aware context packing. Config via .cre/config.yaml. All commands must have --help strings and Rich terminal output. Include pytest stubs for all core modules. Do not build the TUI — stub tui.py only.'

# 5. Bihu Sprint Milestones

| Day | Milestone | Deliverable | Done When |
|---|---|---|---|
| 1–2 | Scaffold | cre init, config, project structure, pyproject.toml | cre --help works |
| 3–4 | L1 Live | ChromaDB ingest + retrieve pipeline | cre ingest wiki/ && cre retrieve 'bio-mech' returns chunks |
| 5–6 | L2 Live | SQLite tiered memory + tier classification | cre compress session.md files facts into tiers |
| 7–8 | L3 Live | Sidecar backends + token budget injector | cre inject 'architecture query' outputs 2000-token block |
| 9–10 | Polish | README, CONTRIBUTING, PyPI prep, v0.1 tag | pip install cre works, GitHub public, Reddit post ready |

## 5.1 v0.2 Backlog (Post-Sprint)

- TUI viewer (Textual) — browse tiers, search live, see token costs
- cre lint — orphan detection, contradiction flagging, stale tier alerts
- cre sync — watch directory, auto-ingest on file changes
- Session auto-compression — watch a log file, compress on session end
- Export formats — output context as JSON, XML, plain text
- Plugin system — custom chunkers, custom tier classifiers

## 5.2 v0.3 Vision

- Native ARIA OS integration — cre inject pipes directly into ARIA Boot Sync
- Browser extension — select text, cre ingest it
- VS Code extension — cre inject into active editor context
- Web dashboard — SaaS layer (if community traction warrants it)

# 6. Open Source Strategy

## 6.1 Launch Plan

5. GitHub public repo — apache/MIT license, clean README with one-line install
6. Reddit post: r/LocalLLaMA + r/ClaudeAI — frame around the five pain points from the thread
7. HackerNews Show HN — 'I built a CLI to stop my LLM prompts from exploding'
8. Dev.to article — 'How I solved context rot with a 3-layer architecture'
9. Nayan beta test — get first external feedback before public launch

## 6.2 README Must-Haves

- One-line install: pip install cre
- 30-second demo GIF (record with asciinema)
- The five problem statements from Reddit (frame the why)
- Architecture diagram (ASCII version from this PRD)
- Comparison table: CRE vs pure RAG vs TunnelVision
- Contributing guide with 'good first issues' labelled

## 6.3 Positioning

*CRE is not another RAG framework. It is the context management layer that should have existed between your knowledge base and your LLM from day one.*

| | |
|---|---|
| **vs LangChain Memory** | LangChain memory is conversation-scoped and opaque. CRE is file-system-native, inspectable, and works across sessions and tools. |
| **vs TunnelVision** | TunnelVision is a SillyTavern plugin. CRE is a general-purpose CLI for any LLM workflow. |
| **vs pure ChromaDB** | ChromaDB is L1 only. CRE adds L2 tiered abstraction and L3 sidecar compression on top. |
| **vs Mem0 / Zep** | Those are SaaS memory products. CRE is local-first, open source, composable, and you own your data. |

# 7. Success Metrics

| Metric | v0.1 Target | v0.3 Target |
|---|---|---|
| Token reduction on inject | >40% vs raw paste | >70% vs raw paste |
| GitHub stars | 50 (post-Reddit launch) | 500 |
| PyPI installs | 100 in first month | 5,000 |
| External contributors | 1 (Nayan) | 10+ |
| Personal usage | Daily, ARIA OS integrated | Default tool in every LLM session |